Perception et Incertitude

INF8422 : Perception Robotique et Intelligence Spatiale

Prof. Pierre-Yves Lajoie

POLYTECHNIQUE MONTRÉAL

Agenda

1. Motivation et Contexte
2. Sources de l'Incertitude
3. Mesurer l'Incertitude
4. Types et Représentation de l'Incertitude
5. Quantifier l'Incertitude
6. Calibration de l'Incertitude
7. Applications en Perception Robotique

Motivation et Contexte

Comprendre → Mesurer → Quantifier → Calibrer → Agir

L'Impact du Bruit

Perception	**Planification**	**Contrôle**
Capteurs, vision, LiDAR	Trajectoires, décisions	Moteurs, actionneurs
Comprendre le monde	*Choisir les actions*	*Exécuter les actions*

Principe Fondamental

Tous les capteurs, sans exception, sont corrompus par du **bruit** (thermique, électrique, dérive mécanique, vibrations, etc.). Mal modélisée, cette incertitude se propage **en cascade** : données distordues $\longrightarrow$ décisions sous-optimales $\longrightarrow$ actions incorrectes.

Objectif du Cours

1. **Identifier les sources** d'incertitude dans un système robotique (matériel, modèle, environnement).
2. **Classifier** les types d'incertitude : aléatorique (données) vs épistémique (modèle).
3. **Mesurer** l'incertitude avec des métriques appropriées : variance, entropie, NLL, ECE.
4. **Quantifier** l'incertitude avec des méthodes d'apprentissage : ensembles, MC Dropout, prédiction conforme.
5. **Calibrer** un modèle pour que ses probabilités soient fiables.
6. **Agir** avec l'incertitude : comparer des décisions, planifier prudemment et explorer pour réduire ce que le robot ignore.

Qu'est-ce que l'Incertitude ?

Avant de quantifier l'incertitude, il faut s'entendre sur ce qu'est une **probabilité**. Deux interprétations coexistent et les deux serviront dans ce cours.

École Fréquentiste

La probabilité est une **fréquence limite** sur des expériences répétées :

$$P(A) = \lim_{n \to \infty} \frac{n_A}{n}$$

- Le paramètre θ est **fixe mais inconnu** ; ce sont les **données** qui sont aléatoires.

École Bayésienne

La probabilité est un **degré de croyance** conditionné à l'information disponible :

$$p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta)\, p(\theta)$$

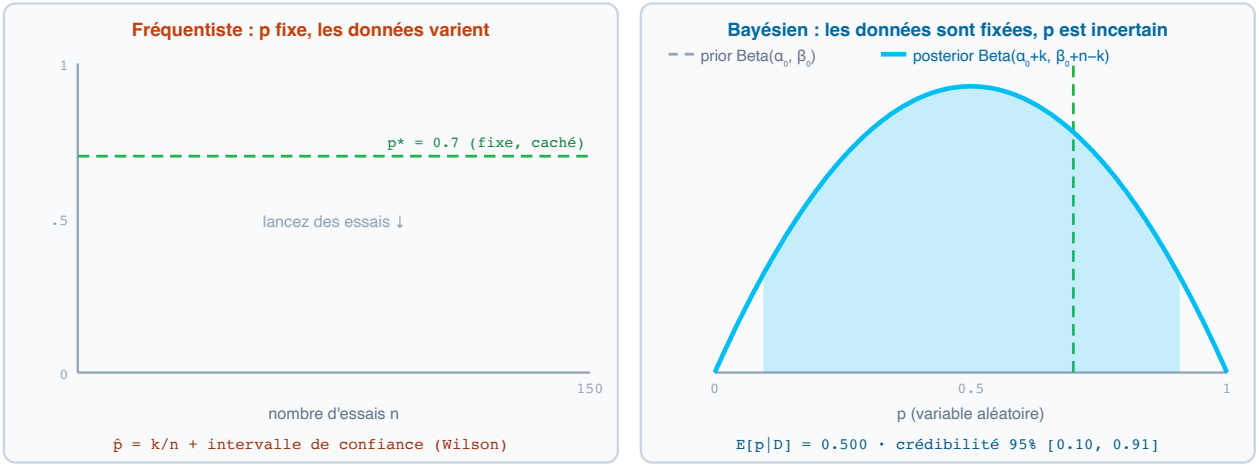
- Le paramètre θ est une **variable aléatoire** ; on conditionne sur les données **observées** (fixes).

Pourquoi c'est important ici

« L'incertitude » d'une prédiction n'a pas le même sens dans les deux écoles : **variabilité sous répétition** (fréquentiste) vs **croyance sur ce qu'on ignore** (bayésien).

Fréquentiste vs. Bayésien — Visualisation Interactive

Le **même flux d'essais** Bernoulli (taux de succès réel $p^* = 0.7$, caché) lu par les deux écoles : à gauche $\hat{p} = k/n$ fluctue autour d'un p^* fixe ; à droite la croyance sur p se concentre.



+1 essai | +10 essais | Reset | force du prior $\alpha_0=\beta_0$ 2 | n = 0, k = 0 succès

L'Incertitude dans la Fusion de Capteurs

Un robot dispose généralement de **plusieurs capteurs** (caméra, LiDAR, GPS, IMU…). Face à des mesures contradictoires, l'incertitude sert de **poids de confiance**.

Fusion Bayésienne

Étant donné deux capteurs indépendants fournissant des mesures z_1 et z_2 sur l'état $\mathbf{x}$:

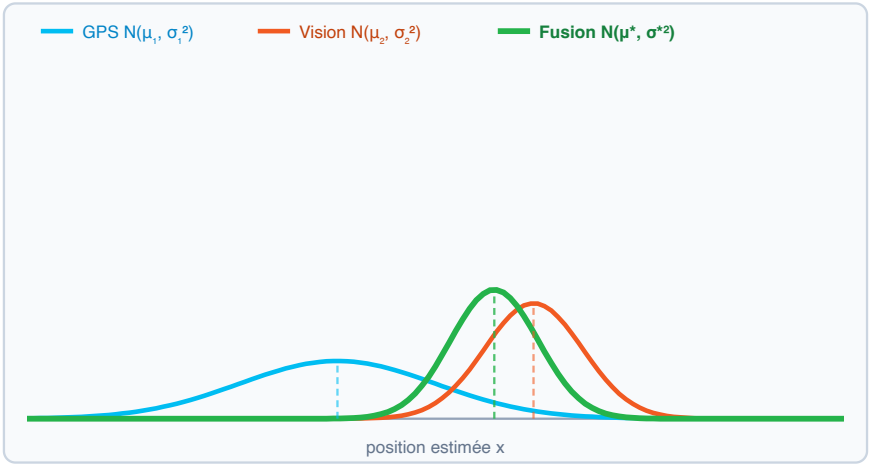
$$p(\mathbf{x} \mid z_1, z_2) \propto \underbrace{p(z_1 \mid \mathbf{x})}_{\text{modèle capteur 1}} \cdot \underbrace{p(z_2 \mid \mathbf{x})}_{\text{modèle capteur 2}} \cdot \underbrace{p(\mathbf{x})}_{\text{prior}}$$

Le capteur le **plus précis** (variance la plus faible) contribue davantage à l'estimation finale.

Intuition : Si le GPS est incertain (faible visibilité des satellites) mais la vision est confiante (repères visuels clairs), la fusion pondère naturellement en faveur de la vision.

Fusion de Capteurs — Visualisation Interactive

Le produit de deux gaussiennes est une gaussienne **plus étroite** : ajustez les précisions et observez qui domine la fusion.



Fusion bayésienne

$1/\sigma^{*2} = 1/\sigma_1^2 + 1/\sigma_2^2$

$\mu^* = \sigma^{*2}(\mu_1/\sigma_1^2 + \mu_2/\sigma_2^2)$

$\sigma_1 = 1.20$ ($\mu_1 = 3.8$)

$\sigma_2 = 0.60$ ($\mu_2 = 6.2$)

$\sigma^* = 0.537$

$\mu^* = 5.72$

$\sigma^* < \min(\sigma_1, \sigma_2) = 0.60$ ✓
La fusion gagne toujours en précision

μ^* penche vers le capteur le plus précis

σ_1 — GPS **1.20** σ_2 — Vision **0.60**

Décider et Prédire sous Incertitude

L'incertitude sur l'état du monde **grandit avec l'horizon temporel**.

Décider : rover sur Mars

- Carte d'élévation **probabiliste** : une *distribution* de cartes, pas une seule.
- Permet d'arbitrer risque de chute vs. consommation d'énergie → décision **risk-aware**.
- Sans incertitude : une seule décision.

Prédire : suivi d'un piéton

- De t_0 à t_1, la « bulle » d'incertitude se **propage et s'agrandit**.
- Outils : échantillonnage de scénarios, POMDPs et planification sensible au risque.
- Actions réductrices : s'approcher, demander de l'aide, trajectoire prudente.

Le problème avec les *Point Estimates*

Un problème des réseaux de neurones

Ces réseaux sont entraînés pour produire une **sortie déterministe unique**: l'estimation ponctuelle θ^* du MLE, sans distribution sur θ. Ils sont souvent **sur-confiants** (*overconfident*) : ils ne savent pas qu'ils ne savent pas.

Conséquences concrètes :

- Véhicule autonome classant une remorque blanche comme « ciel » avec confiance $\longrightarrow$ collision.
- Un réseau confiant à 99% sur une image *adversarielle* (bruit imperceptible à l'œil humain).
- Un détecteur de piétons ignorant qu'il voit un objet jamais vu à l'entraînement.
- Un SLAM neuronal qui perd sa localisation *silencieusement* sans alarme.

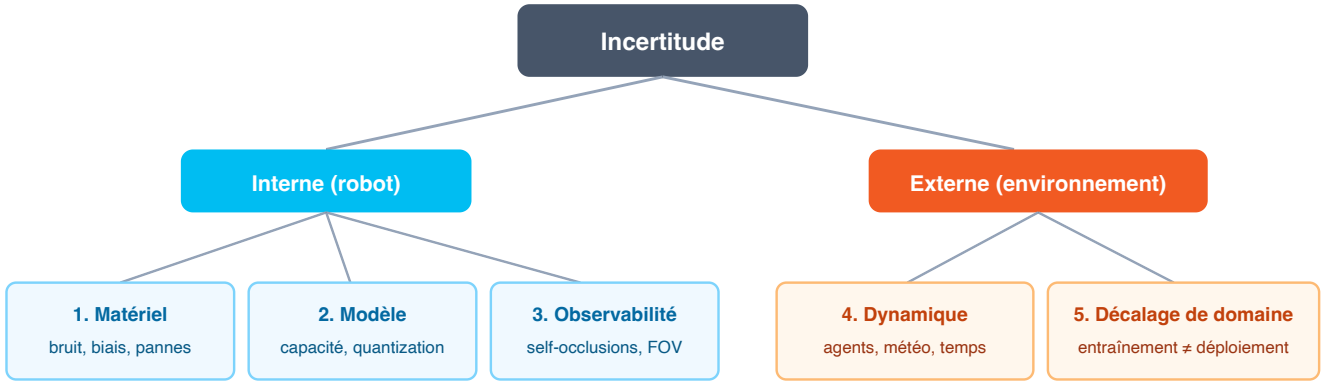
Objectif : Renouer avec le probabilisme et obliger les réseaux à quantifier leur ignorance.

Sources de l'Incertitude

Comprendre → Mesurer → Quantifier → Calibrer → Agir

D'où Vient l'Incertitude ?

L'incertitude en robotique provient de sources **internes** (liées au robot) et **externes** (liées à l'environnement).



Source 1 : Limitations Matérielles

Tous les dispositifs physiques de mesure présentent des **erreurs inévitables**.

Types d'Erreurs de Mesure

- **Erreurs systématiques** : Biais constant (ex : décalage de calibration d'une caméra).
- **Erreurs aléatoires** : Bruit thermique, électronique (ex : bruit gaussien sur un capteur de profondeur).

Exemple composé : Un véhicule autonome tente de localiser un piéton.

- Le **LiDAR** introduit une erreur de distance (± 2 cm).
- Le **GPS** est imprécis en milieu urbain (multi-path, ± 5 m).
- Les deux incertitudes se **composent** : l'incertitude totale sur la position du piéton est bien plus grande que chaque erreur individuelle.

Source 2 : Observabilité Partielle

Un robot ne perçoit jamais le monde dans son **intégralité**.

- **Occlusions** : Un objet caché derrière un autre (piéton derrière une voiture).
- **Conditions visuelles** : Brouillard, obscurité, éblouissement solaire.
- **Encombrement** : Scènes denses avec de nombreux objets superposés.
- **Champ de vue limité** : Un capteur ne voit qu'une portion de l'espace.

Mitigation : Utiliser plusieurs capteurs (modalités complémentaires), planifier des déplacements pour réduire les occlusions, maintenir une mémoire temporelle.

Source 3 : Dynamique de l'Environnement

Le monde dans lequel opère le robot **change dans le temps**.

Éléments dynamiques

- **Agents mobiles** : Piétons, cyclistes, autres véhicules.
- **Objets déplacés** : Meubles dans une maison, objets sur une table.
- **Conditions météo** : Pluie, vent, changement d'illumination.

Conséquences

- La carte construite à t_0 est **périmée** à t_1.
- Les prédictions de trajectoire sont **incertaines**.
- Le contrôle (ex : drone) perturbé par le vent → commandes imprécises.

Incertitude Croissante dans le Temps

Plus l'horizon de prédiction est lointain, plus l'incertitude sur l'état futur est grande. Un robot planifiant à 10 secondes est bien plus incertain qu'un robot planifiant à 1 seconde.

Source 4 : Décalage de Domaine (*Domain Shift*)

Un modèle entraîné sur un ensemble de données peut **échouer** face à des situations non représentées.

Définition

Le **décalage de domaine** survient quand la distribution des données de test $p_{test}(\mathbf{x})$ diffère de la distribution d'entraînement $p_{train}(\mathbf{x})$. Le modèle rencontre des inputs pour lesquels il n'a **aucune expérience**.

Exemples :

- Système de conduite autonome entraîné en Arizona → déployé à Montréal en hiver.
- Modèle de segmentation urbaine → déployé dans une zone **rurale** inconnue.

Exemples Concrets : Trois Domaines Robotiques

Navigation Domestique

- Meubles semi-permanents.
- Personnes dynamiques.
- VLMs/LLMs pour comprendre les instructions humaines.

Manipulation

- Objets occultés ou transparents.
- Masse et friction inconnues.
- Déformabilité incertaine.
- Pose de préhension incertaine.

Robots de Terrain

- SLAM sans GPS (mines, sous-marin, espace).
- Turbulences (drones, AUVs).
- Environnements non structurés.
- Éléments dynamiques (personnes, animaux, autres robots).

L'incertitude dans le SLAM

Aux cours précédents, le SLAM était formulé comme une **estimation probabiliste** : le MAP donne la trajectoire, et la matrice d'information $\Lambda = J^\top \Omega J$ encode en principe toute l'incertitude ($\Sigma = \Lambda^{-1}$).

> En pratique, la plupart des systèmes SLAM déployés n'exploitent que la **moyenne** (l'estimée MAP) et ignorent la covariance. Le robot navigue comme si sa pose et sa carte étaient parfaites.

Pourquoi ce piège existe :

- Récupérer Σ exige de **résoudre le système** : inverser (ou marginaliser) Λ détruit la sparsité qui rend le SLAM temps réel, coûteux en calcul et en stockage sur des graphes de millions de variables.
- Les benchmarks standards (ATE, RPE) ne notent que la précision de la moyenne, pas la qualité de l'incertitude.

Conséquence : une localisation qui dérive *silencieusement*, sans alarme. Les méthodes de ce cours visent précisément à redonner cette **introspection** aux systèmes de perception, classiques comme neuronaux.

Mesurer l'Incertitude

Comprendre → **Mesurer** → Quantifier → Calibrer → Agir

Métriques d'Incertitude

Quantifier l'incertitude est inutile si on ne peut pas l'évaluer et la comparer.

Trois Questions Distinctes

1. **Dispersion :** À quel point la prédiction est-elle étalée ? (Variance, covariance, entropie)
2. **Qualité prédictive :** La distribution prédictive est-elle correcte par rapport à la réalité ? (NLL, Brier)
3. **Distance entre distributions :** Deux distributions sont-elles similaires ? (KL, Hellinger, Wasserstein)

Usages pratiques en robotique :

- Comparer deux méthodes de quantification d'incertitude (ex. Ensembles vs. MC Dropout).
- Évaluer si un modèle est bien calibré sur un jeu de validation.
- Détecter des données hors-distribution (OOD) par seuillage sur une métrique.
- Guider l'exploration active vers les zones d'incertitude maximale.

Variance

La **variance** mesure la *dispersion d'une variable aléatoire* autour de sa moyenne.

Pour une variable X discrète avec N outcomes $\{x_1, \ldots, x_N\}$ et probabilités $\{p_1, \ldots, p_N\}$:

1. Moyenne : $\mu = \mathbb{E}[X] = \sum_{n=1}^{N} p_n x_n$
2. Variance :

$$\sigma^2 = \mathbb{E}\big[(X - \mu)^2\big] = \sum_{n=1}^{N} p_n (x_n - \mu)^2$$

Propriétés importantes :

- $\sigma^2 \geq 0$ (toujours non-négative). $\sigma^2 = 0$ ssi X est déterministe.
- L'**écart-type** $\sigma = \sqrt{\sigma^2}$ est dans les mêmes unités que X (plus interprétable).
- Pour une distribution continue $p(x)$: $\sigma^2 = \int (x - \mu)^2 p(x)\, \mathrm{d}x$.

Limite : Ne mesure la dispersion que d'une *seule* variable (→ **covariance**, diapo suivante). Ne capture pas la forme de la distribution (une bimodale peut avoir la même variance qu'une unimodale).

Covariance

Deux variables d'un même état (ex. x et y d'une pose) ne varient pas indépendamment.

La **covariance** mesure leur *co-variation* :

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)]$$

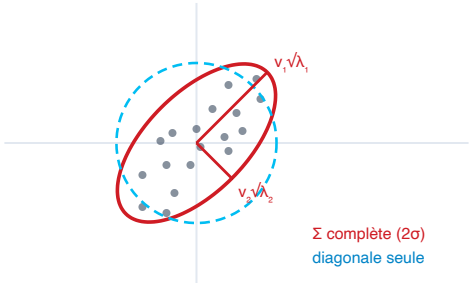
- Signe = sens de la co-variation ; version normalisée : la **corrélation** $\rho = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y} \in [-1, 1]$.

Matrice de covariance pour $\mathbf{x} \in \mathbb{R}^d$:

$$\Sigma = \mathbb{E}\left[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top\right], \qquad \Sigma_{ii} = \sigma_i^2, \quad \Sigma_{ij} = \mathrm{Cov}(x_i, x_j)$$

- Symétrique **semi-définie positive**.
- Géométrie : les vecteurs propres de Σ donnent les axes de l'**ellipse d'incertitude**, les $\sqrt{\lambda_i}$ ses demi-axes.

Pourquoi c'est central en robotique : Σ décrit non seulement l'amplitude de l'erreur, mais aussi ses directions privilégiées – par exemple dans une pose ou une carte SLAM.



Si on ignore les corrélations

Garder seulement la diagonale (pointillés bleus) **sous-estime** l'incertitude dans la direction oblique.

Entropie de Shannon

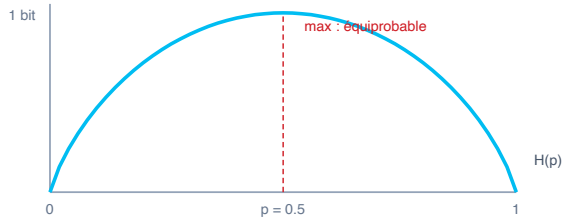
L'**entropie** mesure à quel point les outcomes possibles d'un système sont aléatoires. Contrairement à la variance, elle prend en compte la **forme** de la distribution, pas les valeurs numériques.

Pour une variable X avec N outcomes discrets et probabilités $\{p_1, \ldots, p_N\}$:

$$H(X) = -\sum_{n=1}^{N} p_n \log p_n \qquad (\text{en bits si log base 2, en nats si log naturel})$$

Propriétés :

- $H(X) \geq 0$. $H(X) = 0$ ssi une seule issue est certaine ($p_k = 1$).
- $H(X)$ maximale = $\log N$ quand toutes les issues sont équiprobables.
- Pour un classifieur binaire (p = prob. de la classe positive) : $H = -(p \log p + (1-p)\log(1-p))$.



Negative Log-Likelihood (NLL)

La **NLL** mesure la *qualité d'une distribution prédictive par rapport à la vérité terrain*. C'est la métrique de choix pour évaluer des modèles probabilistes.

En classification : Pour la classe vraie y_t avec probabilité prédite $\hat{p}(y_t)$:

$$\mathrm{NLL}_{\mathrm{class}} = -\log \hat{p}(y_t)$$

Si le modèle est confiant et correct ($\hat{p}(y_t) \approx 1$) : $\mathrm{NLL} \approx 0$. Sinon NLL augmente.

En régression gaussienne : Pour une prédiction $\hat{\mu}$ avec variance $\hat{\sigma}^2$ et vérité y_t :

$$\mathrm{NLL}_{\mathrm{reg}} = \frac{1}{2}\log(2\pi\hat{\sigma}^2) + \frac{(y_t - \hat{\mu})^2}{2\hat{\sigma}^2}$$

> Son espérance est minimisée **si et seulement si** la distribution prédictive $\hat{p}$ est exactement la vraie distribution p.

Distance de Mahalanobis

La **distance de Mahalanobis** mesure la *distance entre un point et une distribution multivariée*, en tenant compte des corrélations.

Pour un point $\mathbf{x} \in \mathbb{R}^d$ par rapport à une distribution $(\boldsymbol{\mu}, \Sigma)$:

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

Cas particuliers :

- $d = 1$: $D_M = |x - \mu| / \sigma$ (distance en nombre d'écarts-types).
- $\Sigma = I$ (identité) : $D_M = \|\mathbf{x} - \boldsymbol{\mu}\|_2$ (distance euclidienne).

Applications en robotique :

- **Détection OOD :** Si $D_M(\mathbf{x}_{\text{test}})$ est grande dans l'espace des features, l'entrée est probablement hors-distribution.
- **Évaluation de régression :** Distance entre la prédiction gaussienne et la vérité terrain.
- **Fusion de capteurs :** Distance de Mahalanobis entre deux estimations pour l'association de données

Comparer des Distributions

Troisième famille de métriques : mesurer l'**écart entre deux distributions** p et q (posterior vs. approximation, features train vs. test, désaccord entre modèles).

Mesure	Formule	Propriétés	Usage typique
Kullback-Leibler	$\mathbb{KL}[p \,\|\, q] = \sum_n p_n \log \frac{p_n}{q_n}$	≥ 0, nulle ssi $p = q$, **non symétrique**	Variational Inference
Jensen-Shannon	$\frac{1}{2}\mathbb{KL}[p \,\|\, m] + \frac{1}{2}\mathbb{KL}[q \,\|\, m]$, $m = \frac{p+q}{2}$	KL symétrisée, bornée	désaccord entre membres d'un ensemble
Wasserstein *Earth Mover Distance* (W_2)	coût minimal de **transport optimal** de p vers q	sensible à la géométrie ; lent à calculer	distance entre histogrammes, entre formes dans des images

Types et Représentation de l'Incertitude

Comprendre → **Mesurer** → Quantifier → Calibrer → Agir

Les Deux Grands Types d'Incertitude

L'incertitude prédictive se divise en deux catégories fondamentales aux propriétés très différentes.

Incertitude Aléatoire

Inhérente aux bruits sur les données

Une partie ne cette incertitude est *irréductible*.

Incertitude Épistémique

Inhérente au modèle

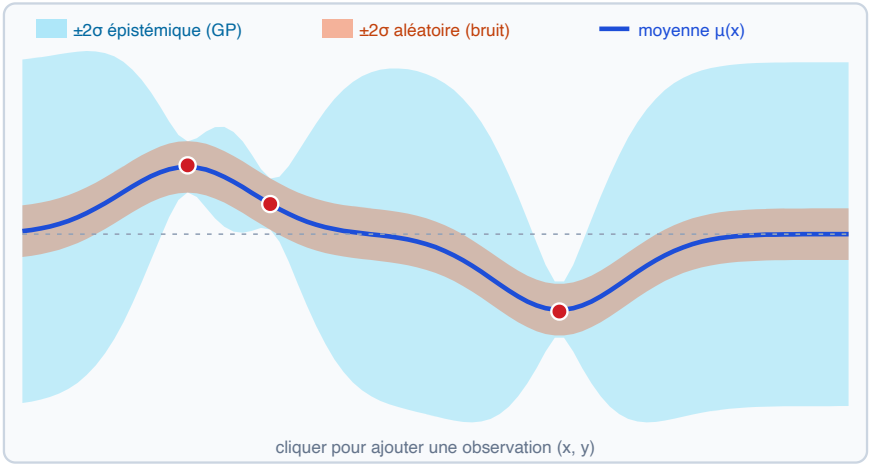
Manque de connaissance, manque de données

Représentation limitée ne pouvant pas encoder toute la complexité des données

Réductible en ajoutant des données, en élargissant le modèle, etc.

Types d'Incertitude – Visualisation Interactive

Régression par processus gaussien : cliquez pour ajouter des observations – la bande **épistémique** s'effondre localement, la bande **aléatoire** ne bouge pas.



Deux incertitudes

Aléatoire — irréductible
Bande orange constante : ajouter des points ne la réduit pas (σ_n = bruit capteur).

Épistémique — réductible
Bande bleue : large loin des observations, s'effondre là où on a des données.

`n = 3 observations`
`σ_aléa = 0.15`
`ℓ = 0.80 (portée)`

σ_aléatoire **0.15** ℓ corrélation **0.80** Reset

Incertitude Aléatoire

Définition

L'**incertitude aléatoire** (aussi appelée *incertitude statistique* ou *data uncertainty*) est due à la **stochasticité inhérente** du phénomène observé. Elle est **irréductible** : même avec une quantité infinie de données, elle ne disparaît pas.

Modèle de mesure : Pour une sortie y étant donné une entrée $\mathbf{x}$ et les vrais paramètres θ^* :

$$y = f_{\theta^*}(\mathbf{x}) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2_{\text{alea}})$$

où ε est le bruit irréductible de la mesure, σ^2_{alea} sa variance.

Exemples en perception robotique :

- Bruit thermique d'un capteur de profondeur (présent même s'il ne bouge pas).
- Ambiguïté d'étiquetage sémantique à la frontière entre deux objets.
- Flou de mouvement dans une image capturée pendant un déplacement rapide.

Incertitude Épistémique

Définition

L'**incertitude épistémique** (aussi appelée *incertitude de modèle* ou *systematic uncertainty*) provient du **manque de connaissance** ou de données. Elle est **réductible** : en collectant plus de données pertinentes, elle diminue.

Modèle de mesure : On cherche les paramètres θ du modèle. L'incertitude épistémique est représentée par la **distribution posterior** :

$$p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta)\, p(\theta)$$

Une distribution $p(\theta|\mathcal{D})$ concentrée = faible incertitude épistémique (beaucoup de données). Une distribution diffuse = haute incertitude épistémique (peu de données).

Exemples en perception robotique :

- Détecteur jamais entraîné sur des objets rares → incertitude haute.
- Carte SLAM d'une zone jamais visitée → incertitude géométrique élevée.
- Modèle de segmentation face à une condition météo inconnue.

Représentations de l'Incertitude

L'incertitude peut être **représentée** sous différentes formes selon le contexte.

Représentations continues

- **Distributions paramétriques** : Gaussienne $\mathcal{N}(\mu, \sigma^2)$, Dirichlet, von Mises-Fisher.
- **Échantillons** (Monte Carlo) : Un ensemble de K prédictions représente le posterior.
- **Fonctions de densité** (flux normalisants, GP).

Représentations discrètes

- **Intervalles / ensembles** : $[y_{low}, y_{high}]$ (prédiction conforme).
- **Ensembles de classes** : {chat, tigre} (classification conforme).
- **Polytopes** et **boîtes de probabilité** (p-box) pour l'ingénierie.

Choix Pratique

Le choix de représentation dépend du compromis **expressivité / coût computationnel**. Les Gaussiennes sont simples et efficaces ; les distributions non-paramétriques (e.g, ensembles) sont plus expressives mais plus coûteuses.

Incertitude Homoscédastique vs. Hétéroscédastique

L'incertitude aléatoire elle-même peut être **constante** ou **variable selon l'entrée**.

Homoscédastique

Variance **constante** : $\sigma^2_{\text{alea}} = \sigma^2$ (indépendant de $\mathbf{x}$).

- Bruit thermique d'un capteur.
- Erreur de quantization uniforme.

Simple à modéliser : un seul scalaire à apprendre.

Hétéroscédastique

Variance **dépendante de l'entrée** : $\sigma^2_{\text{alea}}(\mathbf{x})$.

- Erreur de profondeur stéréo : $\sigma_Z \propto Z^2$ (croît avec la profondeur Z).
- Segmentation : incertitude plus haute aux frontières d'objets.

Le réseau doit **prédire** la variance.

Représenter l'Incertitude Aléatoire : Classification

En classification avec K classes, le réseau produit naturellement une distribution de probabilité.

La couche **Softmax** transforme les logits z_k en probabilités :

$$p(y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad \sum_{k=1}^{K} p(y = k|\mathbf{x}) = 1$$

Mesure d'incertitude aléatoire : L'entropie de la distribution softmax :

$$H = -\sum_{k=1}^{K} p_k \log p_k$$

- $H = 0$: une seule classe à 100% de probabilité → très certain.
- $H = \log K$: distribution uniforme → maximalement incertain.

> **Attention**
>
> Cette entropie est l'incertitude *vue par un seul modèle* θ^* : structurellement **aveugle à l'épistémique** (elle peut être ≈ 0 en zone OOD).

Représenter l'Incertitude Aléatoire : Régression

En régression, une *uncertainty head* ne prédit pas une « confiance » abstraite : elle transforme le réseau en modèle d'une **distribution conditionnelle** $p(y \mid \mathbf{x})$.

Architecture à Double Tête

Un backbone partagé produit $\mathbf{h} = f_\phi(\mathbf{x})$, puis deux têtes prédisent :

$$\hat{\mu}(\mathbf{x}) = g_\mu(\mathbf{h}), \qquad \hat{\sigma}^2(\mathbf{x}) = g_\sigma(\mathbf{h}) > 0$$

- $\hat{\mu}$: la valeur centrale attendue;
- $\hat{\sigma}^2$: l'échelle des erreurs attendues pour **cette entrée**.

La tête de variance apprend l'incertitude **aléatoire hétéroscédastique**. Elle ne détecte pas à elle seule une entrée hors distribution : il faudra lui ajouter une méthode épistémique.

Sortie multivariée : Pour $y \in \mathbb{R}^d$ (ex. une boîte 3D), prédire une covariance $\Sigma(\mathbf{x})$. Elle doit rester semi-définie positive.

Astuce de Cholesky : Prédire la matrice triangulaire inférieure L telle que $\Sigma = LL^\top$:

$$\hat{\Sigma} = \hat{L}\hat{L}^\top, \qquad \mathrm{diag}(\hat{L}) > 0$$

Cela garantit une covariance valide tout en apprenant les corrélations entre sorties.

Décomposition de la Variance Totale

La variance totale d'une prédiction se décompose en deux contributions indépendantes.

Variance Totale

Pour un modèle avec paramètres θ :

$$\underbrace{\mathrm{Var}(y \mid \mathbf{x})}_{\text{variance totale}} = \underbrace{\mathbb{E}_\theta\left[\sigma^2_{\text{alea}}(\mathbf{x}, \theta)\right]}_{\text{variance aléatoire}} + \underbrace{\mathrm{Var}_\theta[\mu_\theta(\mathbf{x})]}_{\text{variance épistémique}}$$

Interprétation pratique :

- **Variance épistémique** = variance des moyennes prédites par différents modèles plausibles.
- **Variance aléatoire** = bruit moyen prédit par ces modèles.

Cette décomposition est exploitée par les **ensembles profonds** et **MC Dropout** pour séparer les deux types.

Quand les Deux Types se Mélangent

En pratique, la séparation stricte aléatoire/épistémique peut être **floue**.

Exemples Ambigus

- **Occlusion :** Un objet est caché. Est-ce de l'aléatoire (physique : l'objet est là mais invisible) ou de l'épistémique (le modèle n'a jamais appris à inférer derrière les occlusions de ce type) ?
- **Frontière sémantique :** Un pixel entre « route » et « trottoir ». Aléatoire (ambiguïté inhérente) ou épistémique (le modèle n'a pas assez d'exemples de frontières) ?

Limites théoriques :

- Les méthodes de type dropout capturent parfois de l'aléatoire au lieu de l'épistémique.
- Les réseaux dédiés à la variance ($\hat{\sigma}^2$) peuvent absorber de l'épistémique.
- La décomposition est imparfaite, mais c'est une abstraction utile.

Résumé : Tableau Comparatif

Type	Cause	Réductible ?	Représentation utile
Aléatoire	Bruit capteur, stochasticité inhérente, ambiguïté des données	**Non**	Distribution $p(y \mid x)$, variance ou covariance
Épistémique	Manque de données, zones non vues, domain shift	**Oui** (plus de données)	Distribution sur les modèles, désaccord prédictif

Message Clé

Pour un système robotique **sûr** : il faut capturer les **deux** types. L'aléatoire pour modéliser fidèlement le bruit capteur, l'épistémique pour détecter les situations inconnues et potentiellement dangereuses.

Quantifier l'Incertitude

Comprendre → Mesurer → **Quantifier** → Calibrer → Agir

Les Systèmes classiques quantifient déjà l'incertitude

Trois piliers de la métrologie en robotique :

📄 Fiche technique (*datasheet*)

Le fabricant mesure le bruit **en usine** et le publie :

- LiDAR : ±3 cm (1σ) à 100 m.
- Caméra ToF : ~1 % de la portée.
- IMU : densité de bruit (°/s/√Hz), *bias instability*.

Toujours avec des **conditions de validité** : portée, température, réflectivité de la cible.

🎯 Calibration

Retirer le **biais systématique** et mesurer le bruit résiduel :

- Mire d'échiquier → RMS de reprojection → σ_{pixel}.
- Étalonnage (certificat, traçabilité).

🧪 Caractérisation (tests)

Mesures répétées en conditions contrôlées → **ajuster le modèle de bruit**

- Histogramme des erreurs → gaussienne ; $\sigma(Z) \propto Z^2$ en stéréo (déjà vu).

> **Il existe même une norme pour ça**
>
> Le **GUM** (*Guide to the Expression of Uncertainty in Measurement*, ISO) codifie tout ce processus.

Pourquoi la Datasheet ne suffit pas

Ce pipeline métrologique repose sur une hypothèse : le bruit du capteur est **stationnaire** et sa physique **connue**.

Malheureusement on opère rarement dans de telles conditions parce que le comportement des capteurs dépend aussi de l'environnement.

	Capteur classique	**Réseau de neurones**
Source d'erreur	physique connue (bruit thermique, quantification)	dépend de l'entrée **et** des données d'entraînement
Stationnarité	stable dans les conditions spécifiées	stable dans le domaine d'entraînement
Caractérisation	en labo / en usine	à refaire pour chaque distribution rencontrée

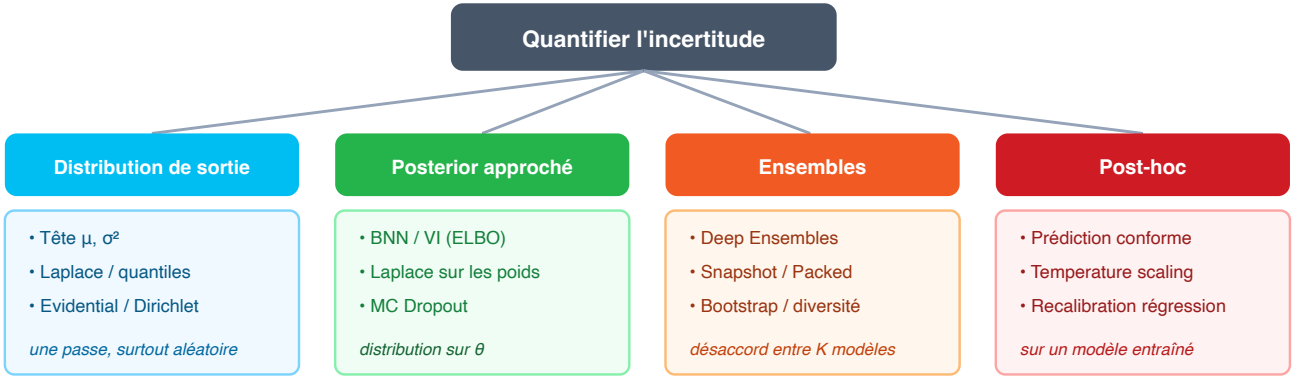
Une performance moyenne sur un jeu de test est une première source d'estimation de la variance, mais elle ne dit rien sur *cette* image, *cette* scène.

Nous ne nous attarderons pas sur l'estimation de l'incertitude nominale de capteurs (datasheet), voir un cours de métrologie pour plus de détails (e.g. MEC8552).

Le focus sera sur l'incertitude produite par les systèmes de perception et d'intelligence artificielle, puis sur son utilisation pour décider et planifier.

Vue d'Ensemble : Taxonomie des Méthodes

Comment produire concrètement une distribution ou un ensemble prédictif ? Quatre familles **complémentaires** :



Combinables : une tête hétéroscédastique peut équiper chaque membre d'un ensemble.

Apprendre l'Hétéroscédasticité : Loss NLL

On ne possède généralement pas de cible « vraie variance ». Les mêmes labels y_i entraînent indirectement **les deux têtes** en maximisant leur vraisemblance.

Negative Log-Likelihood Gaussienne

En supposant $p(y_i|\mathbf{x}_i) = \mathcal{N}(\hat{\mu}_i, \hat{\sigma}_i^2)$, la loss NLL est :

$$\mathcal{L}_{\text{NLL}} = \frac{1}{N}\sum_{i=1}^{N} \underbrace{\frac{\|y_i - \hat{\mu}_i\|^2}{2\hat{\sigma}_i^2}}_{\text{terme de précision}} + \underbrace{\frac{1}{2}\log \hat{\sigma}_i^2}_{\text{régularisation}}$$

Équilibre de la loss :

- Si $\hat{\sigma}_i^2$ est trop petite, une erreur de moyenne coûte très cher.
- Si $\hat{\sigma}_i^2$ est trop grande, le terme $\log \hat{\sigma}_i^2$ la pénalise.
- L'optimum associe donc une petite variance aux cas prévisibles et une grande variance aux cas intrinsèquement ambigus.

Piège pratique

La variance peut temporairement « expliquer » les mauvaises prédictions en gonflant. Une NLL qui baisse ne signifie donc pas nécessairement que la moyenne s'améliore : il faut surveiller les deux séparément.

Quelle sistribution prédire ?

Ajouter une tête revient d'abord à choisir une famille pour $p(y \mid \mathbf{x})$. Cette hypothèse détermine les sorties du réseau et la loss.

Tête probabiliste	Sorties	Loss	Quand l'utiliser
Gaussienne	$\mu,\ \log \sigma^2$	NLL gaussienne	Erreurs symétriques, peu de valeurs extrêmes
Laplace	médiane m, $\log b$	$\lvert y - m \rvert / b + \log b$	Erreurs à queues plus lourdes, cible robuste
Quantiles	$q_{\tau_1}, \ldots, q_{\tau_K}$	*Pinball loss*	Intervalles directs, sans forme paramétrique complète
Gaussienne multivariée	$\boldsymbol{\mu},\ L$	NLL multivariée	Sorties corrélées : pose, boîte 3D, flot optique

Principe de choix

La tête doit refléter la **forme des résidus** et la structure de la sortie. Une variance gaussienne est utile seulement si cette approximation est raisonnable; elle ne représente pas automatiquement toute l'incertitude.

Réf. : Kendall & Gal, 2017.

Entraîner une tête hétéroscédastique

Pseudo-code d'un minibatch

```
h       ← backbone(x)
mu      ← mean_head(h)
s_raw   ← scale_head(h)
s       ← clip(s_raw, s_min, s_max)   # log variance

sq_err  ← (y - mu)²
loss    ← mean(0.5 · exp(-s) · sq_err + 0.5 · s)

zero_grad()
backward(loss)
optimizer.step()
```

Ce qui est réellement appris

- $s = \log \sigma^2$ garantit $\sigma^2 = e^s > 0$.
- e^{-s} pondère chaque résidu par la précision prédite.
- Le gradient traverse les deux têtes **et** le backbone partagé.
- Aucun label de variance n'est requis : la NLL compare la distribution prédite au seul label y.

Alternative stable : prédire $r \in \mathbb{R}$, puis poser $\sigma^2 = \mathrm{softplus}(r) + \varepsilon$.

À l'inférence : $\hat{\sigma} = e^{s/2}$ et, sous l'hypothèse gaussienne, un intervalle approximatif à 95 % est $\hat{\mu} \pm 1.96\hat{\sigma}$.

Stabiliser et vérifier l'entraînement

Recette d'entraînement

1. Entraîner d'abord la moyenne avec MSE ou Huber, variance fixée.
2. Activer la tête d'échelle et optimiser conjointement la NLL.
3. Si la moyenne cesse de progresser, utiliser la β-NLL :

$$\mathcal{L}_\beta = \mathrm{stopgrad}\left[(\hat{\sigma}^2)^\beta\right] \mathcal{L}_{\mathrm{NLL}}, \qquad 0 \leq \beta \leq 1$$

La pondération détachée modifie l'importance des exemples sans offrir un nouveau raccourci à la tête de variance.

Vérifier sur validation

- **Valeur centrale :** MAE ou RMSE de $\hat{\mu}$.
- **Distribution complète :** NLL.
- **Intervalles :** couverture empirique **et** largeur moyenne.
- **Sous-groupes :** répéter selon distance, éclairage, météo et domaine.

Limite à retenir

Cette tête apprend surtout l'aléatoire **dans le domaine d'entraînement**. Pour signaler l'inconnu, la combiner avec un ensemble, MC Dropout ou une autre méthode épistémique, puis la calibrer sur des données séparées.

Réf. : Seitzer et al., 2022.

Le Théorème de Bayes : Fondement

La plupart des méthodes de quantification d'incertitude sont ancrées dans le **théorème de Bayes**.

Pour les paramètres θ d'un modèle et un jeu de données $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ (entrées x, sorties y) :

$$\underbrace{p(\theta \mid \mathcal{D})}_{\text{posterior}} = \frac{\underbrace{p(\mathcal{D} \mid \theta)}_{\text{vraisemblance}} \cdot \underbrace{p(\theta)}_{\text{prior}}}{\underbrace{p(\mathcal{D})}_{\text{evidence}}}$$

Chaque terme :

- $p(\theta)$: croyance **a priori** sur les paramètres (avant de voir les données).
- $p(\mathcal{D}|\theta)$: **vraisemblance** des données étant donné les paramètres.
- $p(\mathcal{D}) = \int p(\mathcal{D}|\theta)\, p(\theta)\, \mathrm{d}\theta$: **evidence** (généralement **intractable**, utilisé pour la normalisation seulement).
- $p(\theta|\mathcal{D})$: **posterior**, notre croyance mise à jour après avoir observé $\mathcal{D}$.

Distribution Prédictive Bayésienne

L'objectif final est de prédire pour une nouvelle entrée $\mathbf{x}^*$.

Distribution Prédictive

En marginalisant sur le posterior des paramètres :

$$p(y^* \mid \mathbf{x}^*, \mathcal{D}) = \int \underbrace{p(y^* \mid \mathbf{x}^*, \theta)}_{\text{vraisemblance prédictive}} \cdot \underbrace{p(\theta \mid \mathcal{D})}_{\text{posterior}} \, \mathrm{d}\theta$$

Cette intégrale capture à la fois l'aléatoire ($p(y^*|\mathbf{x}^*, \theta)$) et l'épistémique ($p(\theta|\mathcal{D})$).

Approximation par échantillonnage : Si on dispose de K échantillons $\{\theta_k\}_{k=1}^K \sim p(\theta|\mathcal{D})$:

$$p(y^* \mid \mathbf{x}^*, \mathcal{D}) \approx \frac{1}{K} \sum_{k=1}^{K} p(y^* \mid \mathbf{x}^*, \theta_k)$$

Le problème est d'obtenir ces échantillons θ_k du posterior, c'est ce que font les différentes méthodes.

Réseaux de Neurones Bayésiens (BNN)

Idée Fondamentale

Au lieu d'un seul poids w_{ij} par connexion, placer une **distribution de probabilité** $q(w_{ij})$ sur chaque poids. Le réseau représente ainsi une distribution de réseaux possibles.

Pour un réseau avec poids $\mathbf{w}$, on apprend une distribution variationnelle $q(\mathbf{w}) = \prod_{ij} \mathcal{N}(\mu_{ij}, \sigma_{ij}^2)$.

Conséquences architecturales :

- Chaque poids w_{ij} est remplacé par **deux paramètres** $(\mu_{ij}, \sigma_{ij}^2)$ → doublement de la mémoire.
- À l'inférence, on **échantillonne** $\mathbf{w} \sim q(\mathbf{w})$ et on effectue le forward pass classique.
- La distribution prédictive est obtenue en répétant ce processus K fois.

Avantage : Cadre mathématiquement rigoureux pour l'incertitude épistémique.

Inconvénients : Mémoire $\times 2$, instabilité numérique, difficultés à l'entraînement.

Variational Inference (VI) et ELBO

Comment entraîner un BNN si le posterior $p(\mathbf{w}|\mathcal{D})$ est intractable ? En utilisant la **Variational Inference**.

Objectif : Trouver $q(\mathbf{w})$ dans une famille paramétrique qui *approche* au mieux $p(\mathbf{w}|\mathcal{D})$, en minimisant $\mathbb{KL}[q(\mathbf{w})||p(\mathbf{w}|\mathcal{D})]$.

Comme on ne peut pas calculer cette KL directement, on maximise l'**Evidence Lower BOund** (ELBO) :

$$\text{ELBO} = \underbrace{\mathbb{E}_{q(\mathbf{w})}[\log p(\mathcal{D} \mid \mathbf{w})]}_{\text{vraisemblance attendue}} - \underbrace{\mathbb{KL}[q(\mathbf{w})||p(\mathbf{w})]}_{\text{régularisation par le prior}}$$

- **Premier terme :** Récompense si q explique bien les données d'entraînement (*fit*).
- **Second terme :** Pénalise si q s'éloigne trop du prior $p(\mathbf{w})$ (régularisation).

Monte Carlo Dropout (MC Dropout)

Le Dropout, habituellement utilisé seulement à l'entraînement pour la régularisation, peut être maintenu actif à l'**inférence** comme approximation bayésienne.

Algorithme :

1. Maintenir le Dropout actif à l'inférence (désactiver des neurones aléatoirement à chaque forward pass).
2. Passer la même entrée $\mathbf{x}$ à travers le réseau N fois.
3. Calculer la moyenne et la variance des N prédictions :

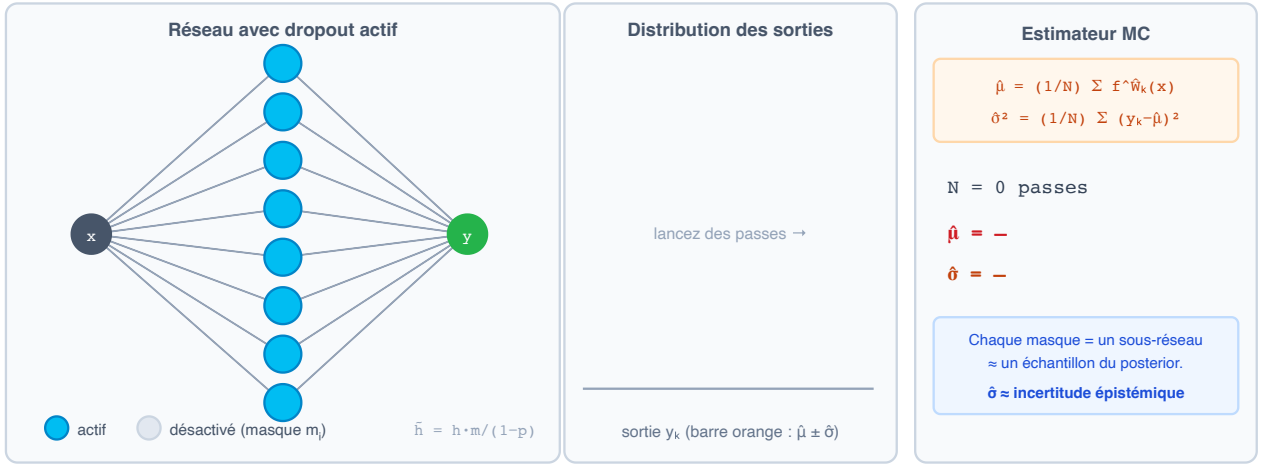
$$\hat{\mu} \approx \frac{1}{N} \sum_{i=1}^{N} f^{\hat{\mathbf{W}}_i}(\mathbf{x}), \qquad \hat{\sigma}^2_{\text{epis}} \approx \frac{1}{N} \sum_{i=1}^{N} \left(f^{\hat{\mathbf{W}}_i}(\mathbf{x}) - \hat{\mu} \right)^2$$

Avantages : Zéro modification architecturale, facile à implémenter.

Inconvénients : Temps d'inférence $\mathcal{O}(N)$, approximation bayésienne faible.

MC Dropout – Visualisation Interactive

Un vrai réseau $1 \to 8 \to 1$: chaque passe tire un **masque** de dropout et exécute un vrai forward pass. La dispersion des sorties estime l'incertitude.



1 passe | 20 passes | Reset | dropout p **0.30**

Ensembles Profonds (*Deep Ensembles*)

Principe : Entraîner K réseaux **indépendants** avec des initialisations de poids différentes.

$$\hat{\mu} = \frac{1}{K} \sum_{k=1}^{K} \mu_k(\mathbf{x})$$

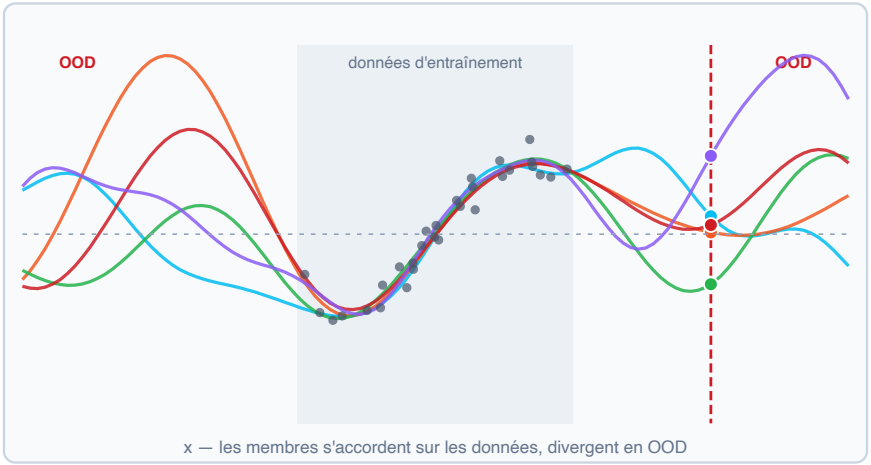
$$\hat{\sigma}^2 = \underbrace{\frac{1}{K} \sum_{k=1}^{K} \left(\mu_k(\mathbf{x}) - \hat{\mu}\right)^2}_{\text{épistémique}}$$

Avantages : Généralement assez précis, parallélisable.

Inconvénients : Coût de mémoire, d'entraînement et d'inférence $K\times$.

Ensembles Profonds – Visualisation Interactive

K régresseurs entraînés différemment (features aléatoires + bootstrap différents) : ils s'accordent sur les données et divergent en zone OOD.



Au point test x*

$$\sigma^{*2} = (1/K)\Sigma\sigma_k^2 + (1/K)\Sigma(\mu_k - \bar{\mu})^2$$

x* = 2.00 $\bar{\mu}$ = 0.14

σ^2_épist = 0.270

K membres **5** Point test x* **2.0** Point ID Point OOD

Variantes d'Ensembles Efficaces

Le coût $K\times$ des ensembles profonds a motivé plusieurs **variantes plus légères**.

Snapshot Ensembles

- Un seul entraînement avec un learning rate **cyclique**.
- Sauvegarder le modèle à chaque minimum local (« snapshot »).
- K snapshots = K membres de l'ensemble.
- Coût d'entraînement: $1\times$ au lieu de $K\times$.

Packed Ensembles

- K sous-réseaux dans une **même architecture**.
- Calcul parallèle sur GPU.
- Coût mémoire réduit.

BatchEnsemble (Wen et al., 2020)

- Poids partagés + facteurs **rang-1** par membre : $W_k = W \odot r_k s_k^\top$.
- Mémoire quasi $1\times$, membres calculés en un seul batch.

Adversarial Training

- Entraîner les membres avec des fonctions de perte qui maximisent leur diversité.

Prédiction Conforme : Apprendre une marge d'erreur

Un modèle prédit $\hat{y}(\mathbf{x})$, mais cette valeur seule ne dit pas **jusqu'où son erreur peut aller**.

Idée en une phrase

On garde des données de **calibration** à part, on observe les erreurs du modèle sur ces données, puis on choisit une marge $\hat{q}$ assez prudente pour entourer les nouvelles prédictions.

Régression

$$\hat{y}(\mathbf{x}^*) \quad \longrightarrow \quad [\hat{y}(\mathbf{x}^*) - \hat{q},\ \hat{y}(\mathbf{x}^*) + \hat{q}]$$

La sortie est une **bande de valeurs plausibles**.

Classification

$$\text{chat} \quad \longrightarrow \quad \{\text{chat},\ \text{tigre}\}$$

La sortie est un **ensemble de classes plausibles**.

La prédiction conforme ne réentraîne pas le modèle : elle **calibre la taille de l'ensemble retourné** à partir de ses erreurs passées.

Split Conformal : Construire la Bande

1. Mesurer les erreurs

Sur n données de calibration, calculer :

$$s_i = |y_i - \hat{y}(\mathbf{x}_i)|$$

2. Choisir une marge prudente

Trier $s_{(1)} \leq \cdots \leq s_{(n)}$, puis :

$$k = \lceil (n+1)(1-\alpha) \rceil, \qquad \hat{q} = s_{(k)}$$

3. Entourer la nouvelle prédiction

$$\mathcal{C}(\mathbf{x}^*) = [\hat{y}(\mathbf{x}^*) - \hat{q},\ \hat{y}(\mathbf{x}^*) + \hat{q}]$$

Exemple : n = 20, objectif = 90 %

$$k = \lceil 21 \times 0.90 \rceil = 19$$

On prend la **19e plus petite erreur** de calibration. Si $s_{(19)} = 0.44$, alors $\hat{q} = 0.44$.

Pour une nouvelle prédiction $\hat{y}(\mathbf{x}^*) = 0.60$:

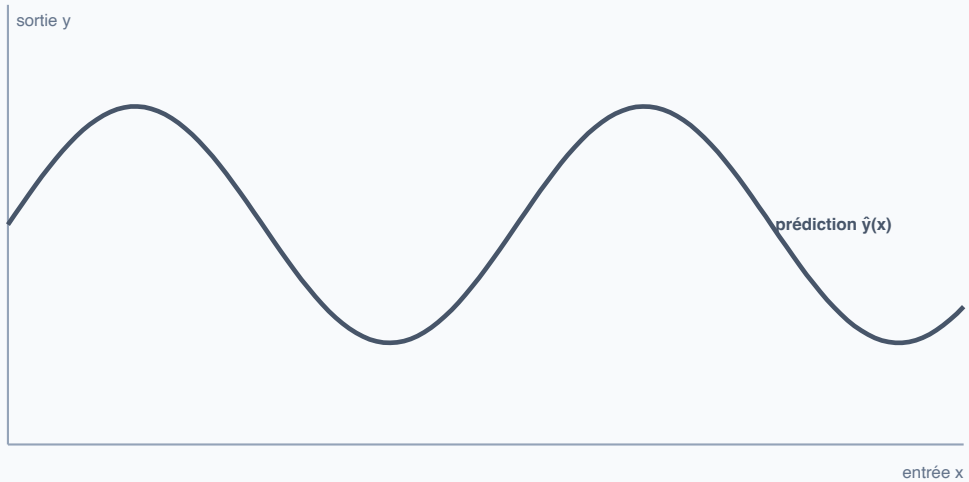
$$\mathcal{C}(\mathbf{x}^*) = [0.16,\ 1.04]$$

Interprétation: La vraie valeur se trouve dans cet intervalle dans 90% des cas.

De la Prédiction à l'Intervalle Conforme

Cliquez sur les quatre étapes : les **erreurs de calibration** deviennent une marge d'incertitude, puis un intervalle pour une nouvelle prédiction.

1 **Prédire** | 2 Mesurer les erreurs | 3 Choisir $\hat{q}$ | 4 Prédire un intervalle



ÉTAPE 1

Le modèle prédit une valeur centrale

$\hat{y}(x)$

La courbe dit ce que le modèle prévoit, mais elle ne donne encore **aucune marge d'erreur**.

Une prédiction seule n'exprime pas sa fiabilité.

Comparaison des Méthodes

Méthode	Épist.	Aléat.	Post-hoc	Coût train	Garanties
Deep Ensembles	✓	≈	Non	$K\times$	Non
MC Dropout	≈	✓	Oui	$1\times$	Non
Laplace	✓	Non	Oui	$1\times$	Non
VI (BNN)	✓	✓	Non	2-$5\times$	Non
Conformal	≈	≈	Oui	$1\times$	**Oui**

≈ : approximation partielle.

Calibration de l'Incertitude

Comprendre → Mesurer → Quantifier → **Calibrer** → Agir

Calibration vs. Netteté (*Sharpness*)

Deux propriétés distinctes et complémentaires caractérisent une bonne distribution prédictive.

Calibration

Cohérence statistique entre les probabilités prédites et les fréquences observées.

Exemple : Si le modèle prédit « voiture » avec probabilité 0.7 sur 100 images, alors environ 70 (en moyenne) doivent être des voitures.

Modèle parfaitement calibré = confiance = précision réelle.

Netteté (Sharpness)

Concentration de la distribution prédictive. Un pic étroit = distribution *informative*.

Exemple : Prédire avec 99% de confiance est plus « net » que 60%.

Une distribution nette **mais mal calibrée** = dangereuse !

L'Objectif

Maximiser la **netteté** tout en garantissant la **calibration**. Les deux ensemble définissent une bonne distribution prédictive pour la sécurité robotique.

Diagramme de Fiabilité (*Reliability Diagram*)

Le **diagramme de fiabilité** est l'outil de visualisation standard pour évaluer la calibration.

Construction :

1. Partitionner les prédictions en M bins selon la **confiance prédite** $\hat{p}$.
2. Pour chaque bin B_m : calculer la **précision réelle** $\mathrm{acc}(B_m)$ (fraction de bonnes prédictions).
3. Tracer $\mathrm{acc}(B_m)$ vs. $\mathrm{conf}(B_m)$ (confiance moyenne du bin).

Lecture du graphe :

- Diagonale parfaite = calibration idéale.
- Au-dessus diagonale = **sous-confiant** (*underconfident*).
- En-dessous diagonale = **sur-confiant** (*overconfident*).

Expected Calibration Error (ECE)

L'**ECE** est la métrique scalaire la plus populaire pour évaluer la calibration.

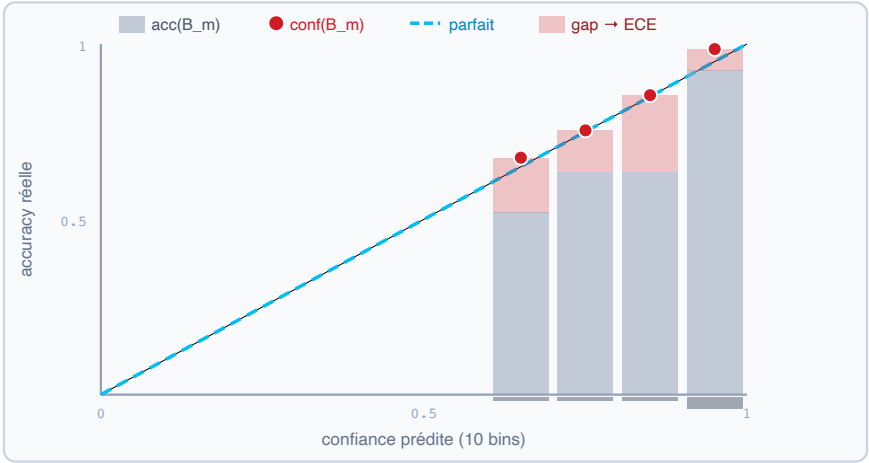
Partitionner les N prédictions en M bins $\{B_m\}_{m=1}^{M}$ selon la confiance prédite. L'ECE est :

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left| \underbrace{\frac{1}{|B_m|} \sum_{i \in B_m} \mathbf{1}[\hat{y}_i = y_i]}_{\mathrm{acc}(B_m)} - \underbrace{\frac{1}{|B_m|} \sum_{i \in B_m} \hat{p}_i}_{\mathrm{conf}(B_m)} \right|$$

- ECE $= 0$: calibration parfaite.
- Pondéré par la taille des bins $|B_m|/N$ → les bins avec plus d'échantillons contribuent davantage.
- **ACE** (*Adaptive CE*) : variante avec des bins de taille égale (même nombre d'échantillon) plutôt que des intervalles uniformes → plus robuste aux distributions déséquilibrées.

ECE – Visualisation Interactive

Le slider de biais décale les **confiances** sans changer les **succès réels** : les gaps colorés sont les contributions de chaque bin à l'Expected Calibration Error (ECE).



Expected Calibration Error

$ECE = \Sigma\ (|B_m|/N) \cdot |acc - conf|$

9.6%

biais de confiance : +0.15

SUR-confiant

Le biais déplace les confiances, pas les succès réels : les points rouges quittent la diagonale.

N = 1000 prédictions, M = 10 bins

Biais de confiance **+0.15**

Sur-confiant (e.g. +0.20) | Calibré (0) | Sous-confiant (e.g. −0.15)

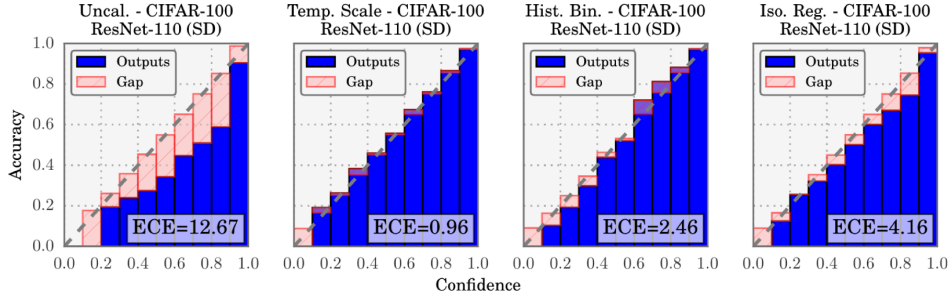
Les réseaux modernes sont mal calibrés

Observation Empirique (Guo et al., 2017)

En comparant des réseaux plus profonds du temps (ResNet, Inception) aux réseaux des années 1990-2000 : la précision top-1 a **augmenté**, mais la calibration s'est **dégradée**. Les réseaux profonds sont **systématiquement sur-confiants**.

Causes probables :

- **Sur-paramétrisation :** Un réseau avec des millions de paramètres peut parfaitement interpoler les données d'entraînement → confiance artificielle.
- **Entraînement prolongé :** Peut mener à du sur-apprentissage.



ResNet-110 sur CIFAR-100 : barres sous la diagonale = sur-confiance. ECE 12.67 % non calibré → 0.96 % après temperature scaling – Guo et al., 2017.

Temperature Scaling

Le **Temperature Scaling** est la méthode de calibration post-hoc la plus populaire par sa *simplicité* et son *efficacité*.

Principe : Diviser les logits $\mathbf{z}$ par un scalaire $T > 0$ avant le softmax :

$$\hat{q}_k = \frac{\exp(z_k/T)}{\sum_{j=1}^{K} \exp(z_j/T)}$$

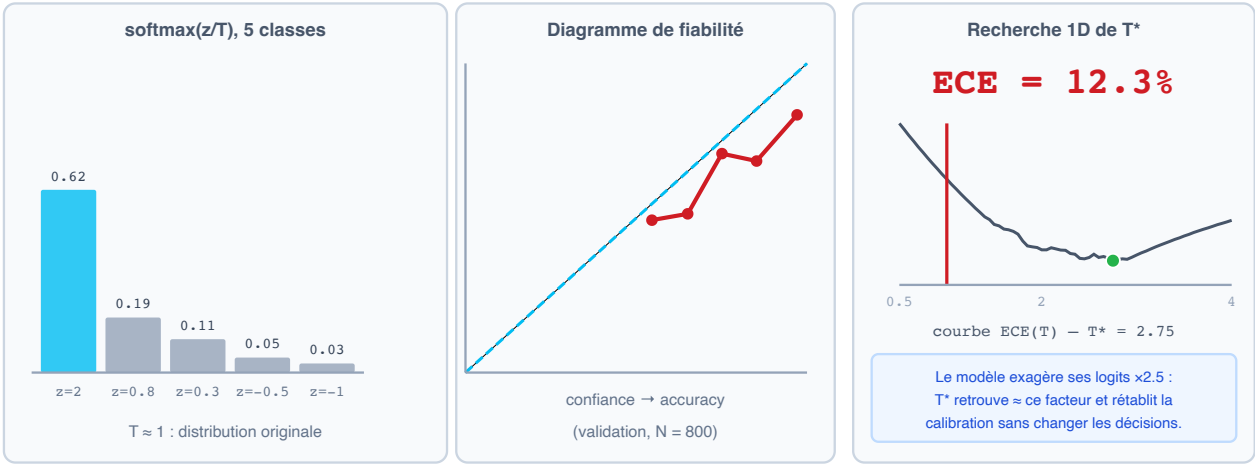
- $T = 1$: identique au modèle original.
- $T > 1$: distribution plus douce (moins confiante).
- $T < 1$: distribution plus piquée (plus confiante).

Optimisation : T est choisi sur un ensemble de **validation** pour minimiser la NLL ou l'ECE.

Avantages : Un seul paramètre, ne modifie pas l'accuracy top-1, post-hoc.

Limites : Même T pour toutes les classes. Échoue sous domain shift.

Temperature Scaling – Visualisation Interactive



Température T **1.00**

Trouver T optimal

T = 1

Histogram Binning

L'**Histogram Binning** est une méthode de calibration **non-paramétrique** qui ajuste directement les probabilités bin par bin.

Algorithme :

1. Partitionner $[0, 1]$ en M bins $\{B_m\}_{m=1}^{M}$ (intervalles de confiance).
2. Sur l'ensemble de calibration, calculer l'accuracy réelle dans chaque bin : $\theta_m = \mathrm{acc}(B_m)$.
3. À l'inférence : si le modèle prédit une confiance $\hat{p}$ dans le bin B_m, retourner θ_m à la place.

Avantages : Non-paramétrique, flexible, peut corriger des erreurs de calibration complexes.

Inconvénients :

- Requiert un grand ensemble de calibration ($n \gg M$).
- Sensible au choix du nombre de bins M.
- Ne généralise pas bien hors des bins observés.

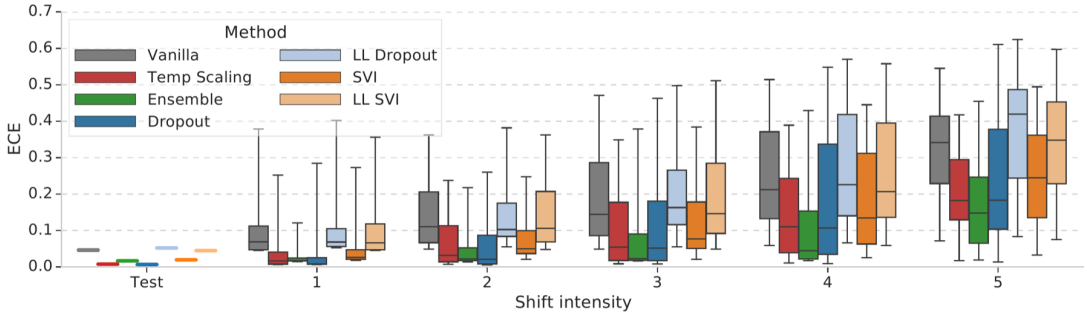
Qualité de Calibration sous Domain Shift

Une question cruciale en robotique : les méthodes de calibration sont-elles **robustes** aux changements de conditions ?

Résultats d'Ovadia et al. (2019)

Étude comparative de plusieurs méthodes (Temperature Scaling, MC Dropout, Ensembles, VI) sous différentes forces de perturbation (corruption, domain shift) :

- La qualité d'incertitude se dégrade avec la force de perturbation pour **toutes** les méthodes.
- Les **Ensembles profonds** et la **VI stochastique** se dégradent le moins vite.
- Le Temperature Scaling seul est **insuffisant** face au domain shift.



ECE sur CIFAR-10 corrompu : l'ECE de toutes les méthodes croît avec l'intensité du shift, mais l'ensemble (vert) reste le plus bas – Ovadia et al., 2019.

Résumé : Quand et Comment Calibrer ?

Protocole Recommandé

1. **Évaluer** toujours la calibration sur un ensemble de validation séparé.
2. **Visualiser** le reliability diagram pour comprendre le sens du biais.
3. **Calculer** l'ECE et la NLL comme métriques.
4. Si ECE $> 5\%$: appliquer une méthode post-hoc.

Quand utiliser le Temperature Scaling :

- Distribution de test similaire au train.
- Budget computationnel très limité.
- Première baseline rapide.

Quand préférer d'autres approches :

- Domain shift attendu → Ensembles.
- Garanties formelles → Conformal.

Principe Fondamental

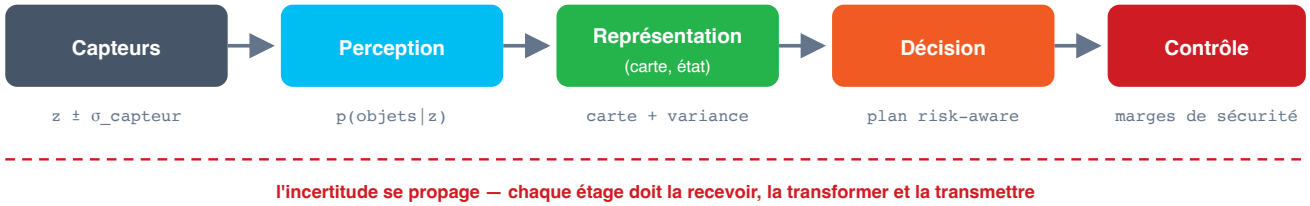
La calibration post-hoc (e.g. temperature scaling) ne remplace pas une **bonne quantification d'incertitude** à la source. Elle améliore un modèle déjà raisonnable ; elle ne peut pas corriger un modèle fondamentalement déficient.

Applications en Perception Robotique

Comprendre → Mesurer → Quantifier → Calibrer → **Agir**

Vue d'Ensemble : L'Incertitude dans le Pipeline Robotique

L'incertitude quantifiée intervient à chaque étape du pipeline robotique.



Perception

- Détection d'objets
- Segmentation
- Estimation de pose

Représentation

- Cartographie
- Localisation
- Modèle du monde

Décision

- Planification
- Contrôle
- Exploration active

Cartographie Probabiliste

La **carte** est la représentation fondamentale du monde pour un robot. Représenter son incertitude est critique.

Occupancy Grids Classiques

- Grille de cellules : $p(\text{occupé}|z_{1:t})$.
- Filtre bayésien récursif par log-odds.

Cartes Probabilistes Avancées

- **GP maps :** Estiment aléatoire **et** épistémique ; continues et différentiables.
- **Bayesian Hilbert Mapping (BHM) :** VI pour scalabilité sur grandes surfaces.

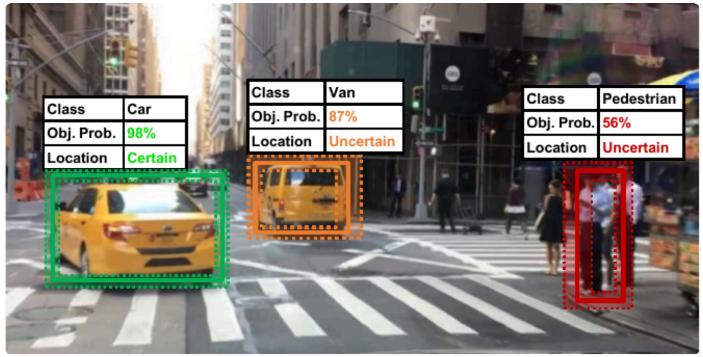
Utilité de l'incertitude de carte : Identifier les zones non visitées (incertitude épistémique haute) pour guider l'exploration, ou les zones dynamiques pour la replanification.

Détection d'Objets Probabiliste

Scénario : un détecteur en ensemble ($K=5$) perçoit un **piéton** en bord de route.

Sortie probabiliste du détecteur :

- Classe : $p(\text{piéton}) = 0.85$ (softmax moyen des membres).
- Distance : $\hat{\mu}_z = 12.0$ m, $\hat{\sigma}_z = 0.8$ m.
- Boîte 3D (x, y, z, w, h, l) : un couple $(\hat{\mu}, \hat{\sigma}^2)$ par paramètre.



Détecteur probabiliste idéal : probabilité de classe et incertitude de localisation (boîtes pointillées) par objet – Feng et al., 2021.

Estimation de Pose 6DoF et Segmentation Sémantique

Mêmes principes, autres têtes de sortie.

Pose 6DoF ($\mathrm{SE}(3)$)

Estimer (x, y, z) + (roll, pitch, yaw) = 6 degrés de liberté.

- **Translation :** Gaussienne multivariée $\mathcal{N}(\boldsymbol{\mu}_t, \Sigma_t)$.
- **Rotation :** Sur $SO(3)$, distributions spécialisées :
  - *von Mises-Fisher* : Distribution sur la sphère.
  - *Bingham* : Pour les quaternions (antipodal).
- **Ensembles** pour pose 6DoF (Wursthorn et al., 2024).

Segmentation Sémantique

Assigner une classe à chaque pixel. Deux types d'incertitude:

- **Incertitude de classe :** Pixel ambigu entre deux catégories (ex : route/trottoir).
- **Incertitude de frontière :** Pixels aux limites entre objets, critique pour la sécurité en conduite autonome.

Détection Hors-Distribution (OOD)

Un système robotique sûr doit savoir **quand il est hors de son domaine de compétence**.

> **Principe**
>
> Si l'incertitude épistémique est élevée pour une entrée $\mathbf{x}^*$, c'est un signal que $\mathbf{x}^*$ est **hors-distribution** (OOD) : le modèle n'a pas appris à traiter des inputs similaires.

Méthodes de détection OOD :

- **Distance de Mahalanobis** dans l'espace des features : si $D_M(\phi(\mathbf{x}^*))$ est grande, $\mathbf{x}^*$ est OOD.
- **Variance d'ensemble** élevée → signal OOD fort.
- **Entropie** de la distribution prédictive.

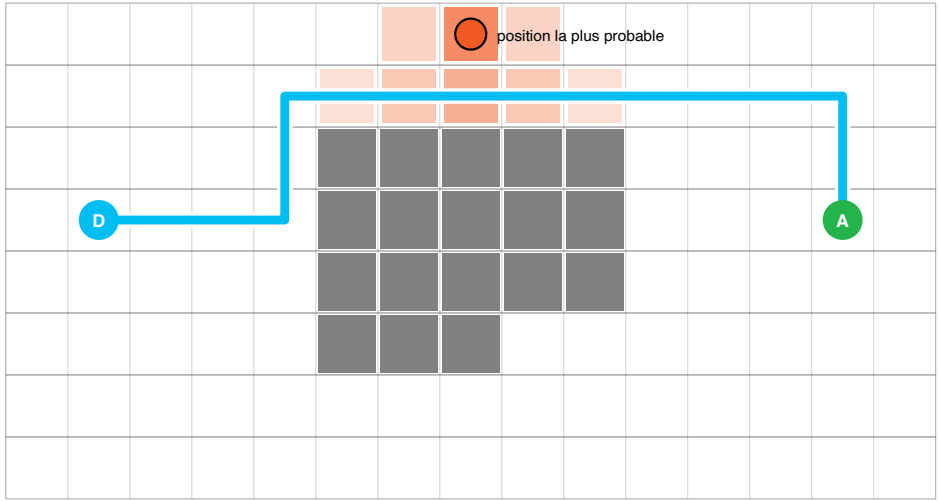
Applications :

- Conduite autonome : détection de kangourous (Nitsch et al., 2021).
- Manipulation : objet inconnu → demander de l'aide.
- Robots spatiaux : pose estimation face à un objet non catalogué.

Idéalement on souhaite que **« Le modèle sache ce qu'il ne sait pas. »**.

Planification sous Incertitude – Visualisation Interactive

La position la plus probable d'un obstacle n'est pas toute l'information. Comparez un trajet court fondé sur cette seule position à un trajet qui tient compte de **toutes les positions plausibles**, puis testez-les dans 30 mondes possibles.



Plan choisi

Longueur : 16 cases

Risque exact :

52 %

Test de 30 mondes

Essais : 0 / 30

Collisions : 0

position plausible

obstacle certain

Position moyenne | Sécurité d'abord | Tester 30 mondes possibles

Le trajet est court, mais il traverse des positions plausibles de l'objet : risque 52 %.

Défis Ouverts et Conclusion

Malgré les avancées considérables, plusieurs **défis ouverts** persistent.

Défis Scientifiques

- **Séparation floue :** Aléatoire vs. épistémique n'est pas toujours clair, dépend du niveau de description.
- **Vérité terrain :** L'incertitude « réelle » d'une image est inconnue. Les simulateurs photo-réalistes aident mais sont imparfaits.
- **Calibration sous shift :** Aucune méthode ne reste parfaitement calibrée hors distribution.

Défis Pratiques

- **Scalabilité :** Les ensembles et BNNs sont trop lents, notamment pour les drones ou la chirurgie temps réel.
- **Grands modèles :** Comment quantifier l'incertitude dans les LLMs et VLMs ?
- **Standardisation :** Pas de protocole universel d'évaluation de l'incertitude.

Message Final

La perception robotique robuste ne se mesure plus seulement en précision (mIoU, AP). La **capacité introspective**, *savoir ce que l'on ne sait pas*, définit la sécurité et la fiabilité d'un système autonome. Quantifier, calibrer, propager puis **agir avec l'incertitude** sont des compétences fondamentales de la perception robotique.

Annexe – Distances entre Distributions

Annexe A1 : Divergence de Kullback-Leibler (KL)

La **divergence KL** mesure la *différence entre deux distributions de probabilité*. C'est un outil central de l'inférence bayésienne.

Pour deux distributions discrètes $p = \{p_n\}$ et $q = \{q_n\}$:

$$\mathbb{KL}[p\|q] = \sum_{n=1}^{N} p_n \log \frac{p_n}{q_n}$$

Propriétés fondamentales :

- $\mathbb{KL}[p\|q] \geq 0$. Nulle si et seulement si $p = q$.
- **Non-symétrique** : $\mathbb{KL}[p\|q] \neq \mathbb{KL}[q\|p]$ en général.
- Rôle central dans la **Variational Inference** : minimiser $\mathbb{KL}[q(\theta)\|p(\theta|\mathcal{D})]$.

La divergence Jensen-Shannon (JSD) est une version symétrisée :

$$\mathrm{JSD}(p, q) = \frac{1}{2}\mathbb{KL}[p\|m] + \frac{1}{2}\mathbb{KL}[q\|m], \quad m = \frac{p+q}{2}$$

Annexe A2 : Distance de Wasserstein

La **distance de Wasserstein** (ou distance de transport optimal) mesure la différence entre distributions en tenant compte de la **géométrie de l'espace**.

La **2-Wasserstein** (W_2) entre p (portée par les points $\{x_i\}$) et q (portée par $\{y_j\}$) :

$$W_2^2(p, q) = \min_{M \geq 0} \sum_{i,j} M_{i,j} \|x_i - y_j\|_2^2, \qquad W_2 = \sqrt{W_2^2}$$

sous les contraintes de marginales : $M\mathbf{1} = p$, $M^\top \mathbf{1} = q$ (M_{ij} = masse transportée de x_i vers y_j).

Intuition : W_2 est le coût minimal pour « déplacer » la distribution p vers q (*Earth Mover's Distance*).

Avantages vs. KL :

- Définie même quand les supports de p et q ne se chevauchent pas.
- Géométriquement sensible : deux Gaussiennes proches ont une petite distance Wasserstein.
- Utilisée pour évaluer les modèles génératifs (GANs) et en apprentissage de mesures d'incertitude.